

ISO26262 Functional Safety Report

Generated At: 2026-06-18 22:14:19

Development Method: **BDD**

Field	Value
Document Template	safety_report
Development Method	BDD
Requirement	What is fault injection testing in ISO 26262?
Related ISO Parts	Part 11 - Semiconductors Part 5 - Hardware Development
Referenced Pages	12, 158, 57, 64, 65, 68, 71, 77, 82, 83

ISO26262 Evidence

Part: Part 11 - Semiconductors

Clause: Unknown

Page: 64

Score: 0.7948

Summary: ■ ISO 26262-11:2018(E) ■ type of safety mechanism including required confidence level; ■ observation points and diagnostic points; ■ fault site, fault list; and ■ workload used during fault injection. In particular, the verification planning describes and justifies: ■ the fault model and the related level of abstraction: ■ As clarified in the following clauses for DFA, digital, analogue and PLD, fault injection can be performed at the appropriate level depending on the fault model being considered, the specific semiconductor technology, feasibility, observability and use case; and NOTE 1 Depending on the purpose, fault injection can be implemented at different abstraction levels (e.g. semiconductor component top-level, part or subpart level, RTL, etc.). A rationale for the abstraction level

Part: Part 11 - Semiconductors

Clause: 5.1.10

Page: 82

Score: 0.779

Summary: ■ ISO 26262-11:2018(E) 5.1.10 Verification using fault injection simulation 5.1.10.1 General As mentioned in 4.8, fault injection is a useful method for semiconductor components. This is especially true for digital circuits for which fault insertion testing of single-event upsets at the hardware level is impractical or even impossible for certain fault models. Therefore, fault injection using design models (e.g. fault injection done at the gate-level netlist) is helpful to complete the verification step. NOTE 1 Fault injection can be used both for permanent (e.g. stuck-at faults) and transient (e.g. single-event upset) faults. NOTE 2 Fault injection is just one of the possible methods for verification, and other approaches are possible. Fault injection utilizing design models can be successful

Part: Part 11 - Semiconductors

Clause: Unknown

Page: 65

Score: 0.7724

Summary: ■ ISO 26262-11:2018(E) EXAMPLE 7 When verifying the diagnostic coverage of a dual-core lock-step, the relevant fault population could be limited to the compared CPU outputs and related fault locations. EXAMPLE 8 When verifying the diagnostic coverage of a software-based hardware test, CPU internal faults are relevant. NOTE 6 Techniques like fault collapsing can also be used to reduce the faults population to prime faults. ■ the fault injection controls, with respect to the related claim in the respective safety analysis; and EXAMPLE 9 Fault injection controls can include the type of fault to be injected, the duration of a transient fault, the number of faults injected in a simulation run, time and location of fault occurrence and the window of observation of the expected action of a safety

Part: Part 5 - Hardware Development

Clause: 4.8

Page: 77

Score: 0.7421

Summary: ■ ISO 26262-11:2018(E) NOTE 7 Due to the complexity of modern digital components (millions of gates), fault injection methods can assist the computation and be used for verification of the amount of safe faults and especially of the failure mode coverage. See 4.8 and 5.1.10 for details. Fault injection is not the only method, and other approaches are possible as described in 5.1.10. 5.1.7.2 How to consider transient faults of digital components 5.1.7.2.1 Failure rate of transient fault As described in ISO 26262-5:2018, 8.4.7, NOTE 2, the transient faults are considered when shown to be relevant due, for instance, to the technology used. They can be addressed either by specifying and verifying a dedicated target “single-point fault metric” value to them or by a qualitative rationale. NOTE A

Part: Part 11 - Semiconductors

Clause: 5.1.2

Page: 83

Score: 0.7322

Summary: ■ ISO 26262-11:2018(E) If properly exercised, methods derived from stuck-at simulations (like N-detect testing, see for example References [35] to [37]) can be applied for verification of non-stuck-at fault models as well. EXAMPLE 3 Since exhaustiveness is not required, the non-stuck-at fault models analysis can be applied to a subset of the digital component subparts selected depending on their possible impact (for example comparators) or on a statistical basis. EXAMPLE 4 For N-detect testing, “properly exercised” means that N different detections of the same fault are guaranteed by the pattern set (i.e. pattern richness). N can range from 5 to 10. NOTE Fault injection can also be used to inject bridging faults (see 5.1.2) in specific locations based on layout analysis or to verify the im

Part: Part 11 - Semiconductors

Clause: Unknown

Page: 57

Score: 0.7059

Summary: ■ ISO 26262-11:2018(E) Table 24 — Systematic dependent failure initiators due to development faults DFI examples Requirement faults Specification errors Implementation faults, i.e. incorrect implementation of functionality Lack or insufficiency of design measures to avoid crosstalk Lack or insufficiency of Latch up prevention measures Wrong configuration Layout faults, such as incorrect routing e.g. over redundant blocks, insufficient insulation, insufficient separation or isolation, insufficient EMI shielding Temperature due to heating of power consuming parts of the die Temperature gradients causing mismatches within sensitive measurement circuitry Measures to prevent dependent failures from violating the safety goal Monitors (e.g. protocol checkers) Measures to prevent the occurrence

Part: Part 11 - Semiconductors

Clause: 5.1.7

Page: 158

Score: 0.6988

Summary: ■ ISO 26262-11:2018(E) Annex C (informative) Examples of quantitative analysis for a digital component C.1 Description The following is an example of a quantitative analysis using the method described in 5.1.7. NOTE 1 Numbers used in this example (e.g. failure rates, amount of safe faults and failure mode coverage) are examples. They can vary from architecture to architecture. NOTE 2 The following examples divide a portion of the digital component into the subparts level. As discussed in 5.1.7, the necessary level of detail can depend on the stage of the analysis and on the safety mechanisms used. NOTE 3 The following examples use the quantitative approach to compute a dedicated target “single-point fault metric” value for transient faults. As discussed in 5.1.7.2, transient faults can be

Part: Part 5 - Hardware Development

Clause: 4.8

Page: 68

Score: 0.6977

Summary: ■ ISO 26262-11:2018(E) Table 28 — Hardware integration tests to verify the completeness and correctness of the implementation of the hardware safety requirements at semiconductor level Method Interpretation at semiconductor level Functional testing Can be covered by pre-silicon verification techniques Electrical testing Can be covered by post-silicon verification techniques, limited to hardware safety requirements that can be verified at that level Fault injection testing See 4.8 Concerning Table 27, the use of the word “test case” is applied somewhat differently between systems and semiconductor components. Semiconductor components are tested in two ways: ■ post-silicon verification focuses on correct integration and freedom from systematic faults and is applied to a small subset of d

Part: Part 11 - Semiconductors

Clause: 5.1.2

Page: 12

Score: 0.6953

Summary: ■ ISO 26262-11:2018(E) The failure mode distribution is correlated with the fault models illustrated in Figure 3. EXAMPLE If a failure mode is caused X % by stuck-at faults and Y % by shorts, and if a safety mechanism only covers stuck-at faults with a coverage of Z %, then the claimed diagnostic coverage is $X \% \times Z \%$. In the context of a semiconductor component, relevant fault models are identified based on the technology and circuit implementation. NOTE 1 See 5.1.2 for further details on fault models for digital components and 5.1.3 for memories. NOTE 2 Typically it is not possible to evaluate every possible physical fault individually due to the number of faults and required level of detail. 4.3.2 Failure modes A failure mode is described at a level of detail commensurate with the safet

Part: Part 11 - Semiconductors

Clause: Unknown

Page: 71

Score: 0.6933

Summary: ■ ISO 26262-11:2018(E) Table 30 — Example of failure modes for digital components Part/subpart Function Aspects to be considered for Failure modea Central Processing Unit (CPU) Execute given instruction flow according to given In- struction Set Architecture. CPU_FM1: given instruction flow(s) not executed (total omission) CPU_FM2: un-intended instruction(s) flow executed (commission) CPU_FM3: incorrect instruction flow timing (too early/late) CPU_FM4: incorrect instruction flow result CPU_FM1 can be further refined if necessary into: ■ CPU_FM1.1: given instruction flow(s) not executed (total omission) due to program counter hang up ■ CPU_FM1.2: given instruction flow(s) not executed (total omission) due to instruction fetch hang up CPU Interrupt Handler circuit (CPU_INTH) Execute interrupt

Hazard / ASIL

Semiconductor component fails to detect faults due to inadequate safety mechanisms

ASIL: ASIL-B

Safety Goal / FSR / TSR

Safety Goal:

Detect critical faults in semiconductor components to prevent unsafe system behavior

FSR:

FSR-001

TSR:

TSR-001

BDD Scenario / Given-When-Then

TC-001 - Verify detection of stuck-at faults in CPU

Given: CPU with hardware lock-step safety mechanism

When: Stuck-at fault is injected into the program counter

Then: CPU fails to execute intended instruction flow

Expected Action: System triggers fault isolation and safe state

Verification Method: Simulation-based fault injection testing

Methodology: BDD

Traceability

Development Method: BDD

Requirement: REQ-001

Hazard: HZ-001
Safety Goal: SG-001
FSR: FSR-001
TSR: TSR-001
Test Cases: TC-001

Trace Chain:
REQ-001 → HZ-001 → SG-001 → FSR-001 → TSR-001 → TC-001